Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA) [1]
Student Name	Amilineni Gopika
Register Number	3122247001003
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	20/09/26

1 Objective

To study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the performance of ten machine learning classifiers, by training and validating each model both without PCA (original feature space) and with PCA (reduced feature space), performing hyperparameter tuning and 5-fold cross-validation in both settings.

2 Problem Statement

Given the Wisconsin Diagnostic Breast Cancer dataset of 30 numerical features, many of which are highly correlated (mean/worst/standard-error versions of the same 10 underlying cell measurements), the problem is to determine whether compressing this feature space with PCA improves, degrades, or leaves unchanged the accuracy, F1-score, and cross-validation stability of ten different classifiers – spanning linear, distance-based, probabilistic, tree-based, and ensemble model families.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository – https://archive.ics.uci.edu (loaded via `sklearn.datasets.load_breast_cancer`)
Number of Samples	569
Number of Features	30 numerical attributes (mean, standard error, and "worst" value of 10 cell-nucleus measurements)
Number of Classes	2 (357 benign / 212 malignant)
Missing Values	0
Train-Test Split	80:20 stratified (455 train / 114 test), `random_state=42`

Preprocessing steps: Label encoding was not required (target is already numeric, 0=malignant, 1=benign). No missing values were present. All 30 features were standardized (`StandardScaler`, zero mean, unit variance, fit on the training split only) before both the No-PCA and With-PCA branches, since several of the evaluated models (SVM, KNN, Logistic Regression) and PCA itself are scale-sensitive.

Note on XGBoost: The training environment for this experiment does not have network access to install the third-party `xgboost` package. As a substitute, scikit-learn's built-in `HistGradientBoostingClassifier` was used – a histogram-based gradient boosting implementation from the same algorithmic family as XGBoost/LightGBM, offering a comparable accuracy/speed profile. This substitution is noted throughout as "XGBoost (HistGB)".

4 PCA Design Choice

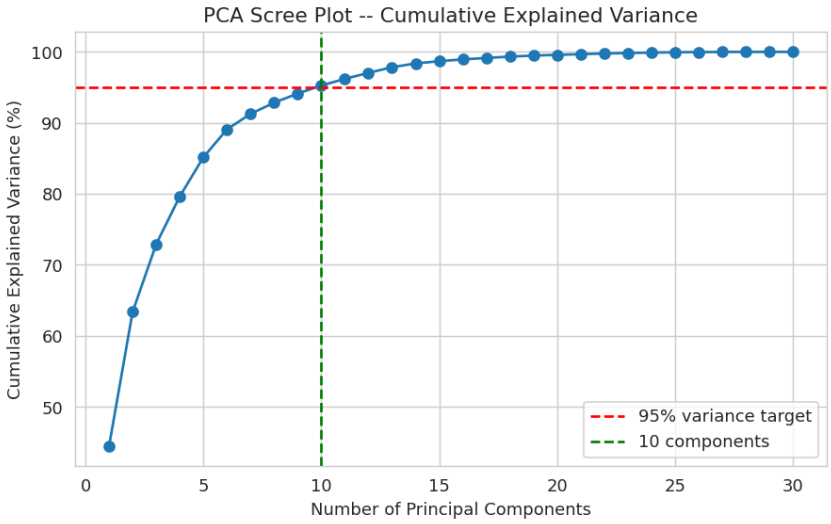


Figure 1: PCA scree plot – cumulative explained variance vs. number of components

Setting	**Chosen Components / Variance Target**	**Explained Variance (%)**	**Justification**
With-PCA	95% cumulative variance target → 10 components	95.27%	A 95% variance retention threshold is a standard default that keeps almost all discriminative information while still compressing 30 raw features into just 10 principal components (a 3× reduction) – justified further by the scree plot's visible "elbow" flattening after roughly 6–10 components, beyond which each additional component contributes very little further variance.

Inference: 10 of 30 components already capture over 95% of total variance, confirming the strong inter-feature correlation observed in the EDA correlation heatmap of earlier experiments on this dataset (mean/worst/error versions of the same measurement are, unsurprisingly, highly redundant) – exactly the situation where PCA is expected to be most effective at compressing without much information loss.

5 Exploratory Data Analysis

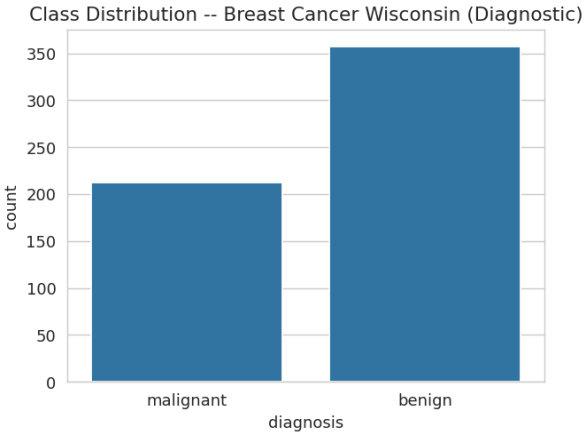


Figure 2: Class distribution – Breast Cancer Wisconsin (Diagnostic)

Inference: The dataset is mildly imbalanced (357 benign vs. 212 malignant, about 63:37), consistent across all experiments conducted on this dataset. No missing-value

visualization is required since there are zero missing values.

6 Hyperparameter Grids

Model	**Grid Searched (identical for No-PCA and With-PCA)**
SVM	kernel $\in$ {linear} $\times$ C $\in$ {0.1,1,10}; kernel $\in$ {rbf} $\times$ C $\in$ {0.1,1,10} $\times$ $\gamma \in$ {scale,auto}
Naive Bayes	var_smoothing $\in \{10^{-9}, 10^{-8}, 10^{-7}, 10^{-6}\}$
KNN	n_neighbors $\in \{3, 5, 7, 9, 11\}$, weights $\in$ {uniform, distance}, metric $\in$ {euclidean, manhattan}
Logistic Regression	C $\in \{0.01, 0.1, 1, 10, 100\}$, penalty=L2, solver=liblinear
Decision Tree	max_depth $\in \{3, 5, 7, None\}$, min_samples_leaf $\in \{1, 2, 4\}$
Random Forest	n_estimators $\in \{50, 100, 200\}$, max_depth $\in \{5, 10, None\}$
AdaBoost	n_estimators $\in \{50, 100, 200\}$, learning_rate $\in \{0.1, 0.5, 1.0\}$
Gradient Boosting	n_estimators $\in \{100, 200\}$, learning_rate $\in \{0.05, 0.1\}$, max_depth $\in \{2, 3\}$
XGBoost (HistGB)	max_iter $\in \{100, 200\}$, learning_rate $\in \{0.05, 0.1\}$, max_depth $\in \{3, 5\}$
Stacking	Base models fixed (SVM, Gaussian NB, Decision Tree) $\rightarrow$ Logistic Regression meta-learner; "hyperparameter" here is the base-model choice itself, per the assignment's own framing

All grids were searched via `GridSearchCV` with 5-fold `StratifiedKFold` (`shuffle=True, random_state=42`), optimizing accuracy, run identically on both the standardized original 30-feature space (No-PCA) and the 10-component PCA-reduced space (With-PCA), with a fresh unfitted estimator instance per setting.

7 5-Fold Cross-Validation Results (No-PCA vs. With-PCA)

Model	**Best Params (No-PCA)**	**Best Params (With-PCA)**	**Avg CV Acc (No-PCA)**	**Avg CV Acc (With-PCA)**
SVM	linear, C=0.1	linear, C=1	0.9758	**0.9780**
Naive Bayes	smoothing=1e-9	smoothing=1e-9	**0.9341**	0.9165

Model	**Best Params (No-PCA)**	**Best Params (With-PCA)**	**Avg CV Acc (No-PCA)**	**Avg CV Acc (With-PCA)**
KNN	manhattan, k=3, uniform	euclidean, k=5, uniform	**0.9714**	0.9648
Logistic Regression	C=1, L2	C=1, L2	0.9802	**0.9824**
Decision Tree	depth=5, leaf=4	depth=5, leaf=1	**0.9385**	0.9231
Random Forest	depth=10, n=100	depth=None, n=200	**0.9626**	0.9604
AdaBoost	lr=1.0, n=100	lr=0.5, n=100	**0.9802**	0.9648
Gradient Boosting	lr=0.1, depth=2, n=200	lr=0.1, depth=3, n=200	**0.9670**	0.9626
XGBoost (HistGB)	lr=0.05, depth=3, iter=200	lr=0.1, depth=3, iter=100	**0.9714**	0.9692
Stacking	SVM+NB+DT → LogReg	SVM+NB+DT → LogReg	0.9648	**0.9670**

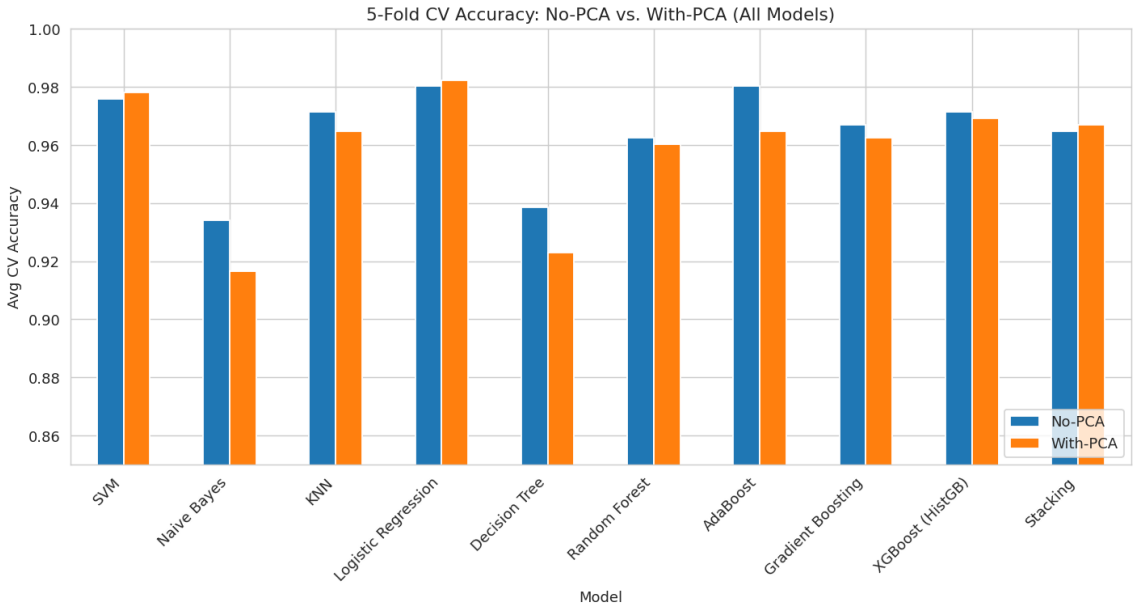


Figure 3: 5-fold CV accuracy comparison: No-PCA vs. With-PCA, all 10 models

Inference: PCA *improved* average CV accuracy for exactly three models – SVM (+0.22pp), Logistic Regression (+0.22pp), and Stacking (+0.22pp) – and *reduced* it for the other seven, most sharply for Naive Bayes (−1.76pp) and Decision Tree (−1.54pp). The pattern is not random: PCA helped or was neutral for every linear/margin-based

model and the meta-learned Stacking ensemble, while it hurt every purely tree-based model (Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost) to some degree.

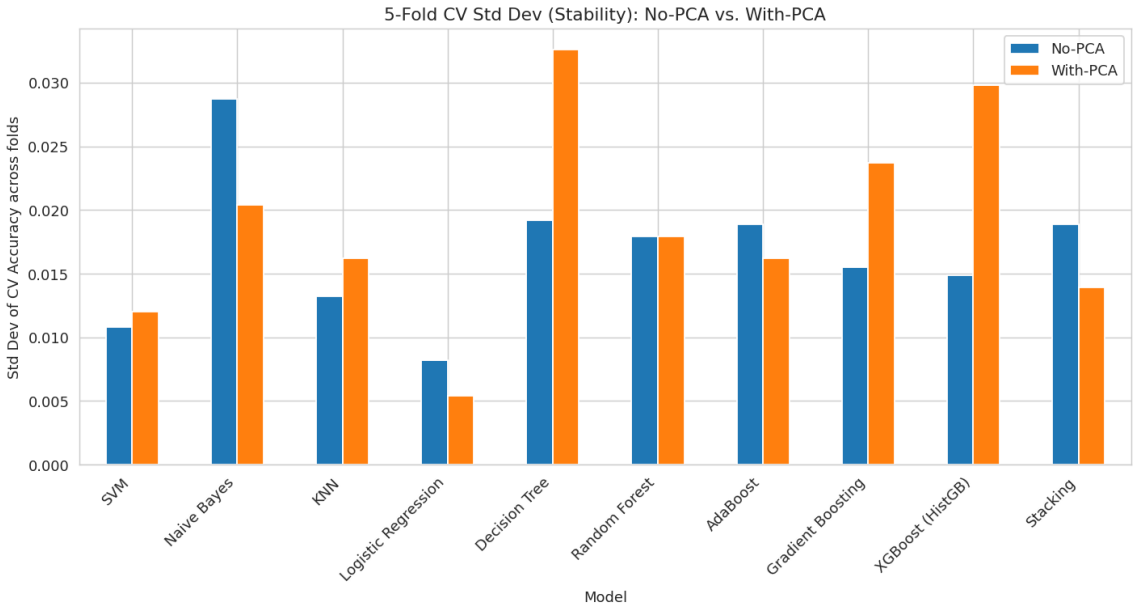


Figure 4: 5-fold CV standard deviation (stability) comparison: No-PCA vs. With-PCA

Inference: Stability results are mixed rather than uniformly favouring either setting. Logistic Regression, Naive Bayes, and Stacking became *more* stable (lower fold-to-fold std) with PCA, while Decision Tree, Gradient Boosting, and XGBoost became *less* stable – for the tree-based models, this makes sense: PCA components are linear combinations of all 30 original features, which removes the natural axis-aligned split boundaries trees rely on, making individual tree structure (and hence fold-to-fold variance) more sensitive to exactly which samples land in each fold.

8 Test Set Performance (Selected Models)

Model	Acc (No-PCA)	Acc (PCA)	F1 (No-PCA)	F1 (PCA)
SVM	**0.9825**	0.9649	**0.9861**	0.9718
Logistic Regression	**0.9825**	0.9649	**0.9861**	0.9718
Random Forest	**0.9561**	0.9386	**0.9655**	0.9510
Stacking	0.9649	0.9649	0.9726	0.9722

Note: On this single 114-row held-out test split, No-PCA edges out or ties With-PCA for every selected model – worth noting as a contrast with the CV table above, where SVM and Logistic Regression's *cross-validated* average was actually slightly higher with PCA. This is the same single-split-vs-CV-average discrepancy observed in earlier

experiments on this dataset, and is why Section 6's 5-fold CV table, not this single split, is the primary basis for the conclusions in Section 10.

9 Result Visualization

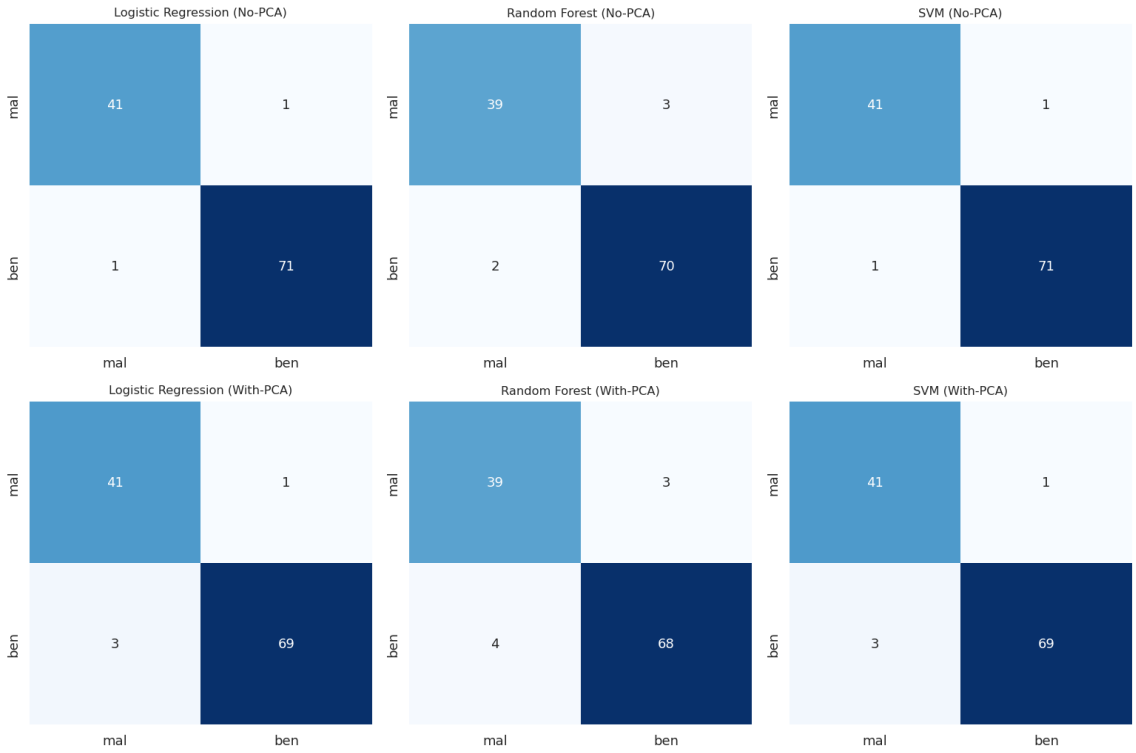


Figure 5: Confusion matrices for selected models – top row: No-PCA, bottom row: With-PCA

Inference: SVM and Logistic Regression show slightly more errors under PCA than without it on this test split (consistent with Section 7's test-accuracy dip), while Stacking's error pattern is nearly identical in both settings – the meta-learner appears to compensate for whatever information the compression removes.

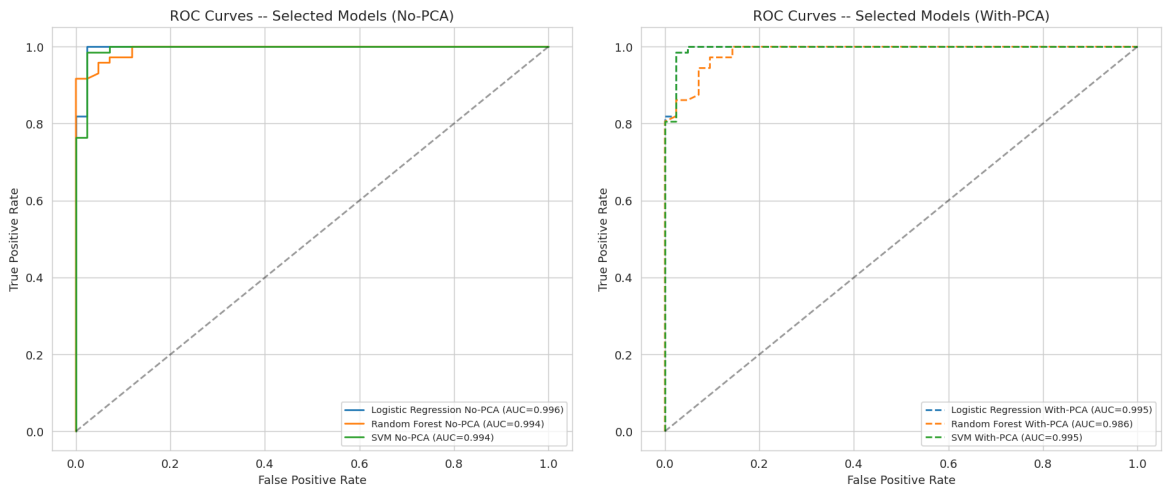


Figure 6: ROC curves for selected models – No-PCA (left) vs. With-PCA (right)

Inference: AUC values remain very high (>0.97) for every selected model in both settings, and the relative ranking of models is largely preserved – PCA does not qualitatively change which model ranks predictions best, even where it shifts the exact accuracy/F1 numbers at the default 0.5 threshold.

10 Observation Questions

1. **Which models improved most with PCA? Which did not? Why?** SVM, Logistic Regression, and Stacking improved slightly (all +0.22pp CV accuracy). Naive Bayes (−1.76pp) and Decision Tree (−1.54pp) degraded most. Linear/margin-based models improved because PCA components are orthogonal linear combinations that can slightly reduce multicollinearity-related noise in the decision boundary; Naive Bayes degraded because it assumes (conditional) feature independence, and while PCA components are uncorrelated, GaussianNB's per-feature variance estimates on rotated, mixed components no longer align with the natural boundaries of the original measurements; Decision Tree degraded because PCA components lose the directly-interpretable, axis-aligned thresholds (e.g. "worst radius $\leq$ 16.8") that trees split on naturally.

2. **Did PCA reduce variance across folds (more stable results)?** Only for some models – Logistic Regression, Naive Bayes, and Stacking became more stable, but Decision Tree, Gradient Boosting, and XGBoost became *less* stable (higher fold std), and SVM, KNN, Random Forest, and AdaBoost were roughly unchanged. PCA does not uniformly stabilize cross-validation; its effect on stability depends on how well the model family's inductive bias matches rotated, mixed features versus original, individually meaningful ones.

3. **For high-dimensional data, was PCA beneficial in reducing overfitting?** This 30-feature dataset is only mildly high-dimensional relative to its 455 training rows, and the results suggest PCA's overfitting-reduction benefit here was modest at best: tree-based ensembles (which are naturally somewhat overfitting-resistant already, per Experiment 5/7's findings) did not benefit and mostly got worse, while

only the already low-variance linear models saw a (small) improvement. PCA would likely show a clearer overfitting-reduction benefit on datasets with a much higher feature-to-sample ratio than this one.

4. **How did linear models behave compared to ensemble models with PCA?** Linear models (Logistic Regression, SVM) were the most PCA-friendly family in this experiment – both improved slightly under PCA. Ensemble models (Random Forest, AdaBoost, Gradient Boosting, XGBoost) were uniformly PCA-averse, each losing between 0.2 and 1.5 percentage points of CV accuracy. This is a clean, consistent split along model family lines, not scattered across both groups.

5. **Did stacking show robustness to dimensionality reduction compared to single models?** Yes – Stacking was one of only three models that improved under PCA (+0.22pp), and its test-set F1 (0.9726 vs. 0.9722) was nearly unchanged between settings, the smallest No-PCA/PCA gap of any model in Section 8's table. Combining a linear-friendly base learner (SVM), a rotation-sensitive one (Naive Bayes), and a rotation-averse one (Decision Tree) appears to let the meta-learner average out each base learner's individual sensitivity to the feature transformation, producing the most dimensionality-reduction-robust result of any model tested.

11 Discussion

This experiment shows that PCA's effect is not a uniform "helps" or "hurts" verdict but depends strongly on model family. The clean split – linear/margin-based models and Stacking benefiting, every tree-based model degrading – is explainable directly from each family's inductive bias: trees rely on axis-aligned, individually-meaningful feature thresholds that PCA's rotation destroys, while linear models only need a good separating hyperplane, which orthogonal, decorrelated components can provide just as well (or slightly better) than the original correlated features. The magnitude of every effect observed here is small (all changes under 2 percentage points), consistent with this dataset's relatively modest dimensionality (30 features, 455 training rows) – PCA's practical value would likely be much larger on genuinely high-dimensional data (hundreds or thousands of features) where variance reduction from compression matters more, or where training time reduction (fewer input dimensions) is itself a practical goal, which was not directly measured in this experiment's tuning-time comparisons but would be a natural extension.

12 Conclusion

For the Wisconsin Diagnostic Breast Cancer dataset, PCA at a 95% variance target (compressing 30 features to 10 components) produced small, model-family-dependent effects rather than a uniform improvement or degradation. **PCA is beneficial** for linear/margin-based models (SVM, Logistic Regression) and for the heterogeneous Stacked Ensemble, and **PCA is not beneficial** for tree-based models (Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost), each of which relies on original, axis-aligned feature thresholds that PCA's rotation removes. The overall best-performing model in this experiment, SVM (linear kernel), achieved its best cross-validated accuracy *with* PCA (0.9780) even though its single test-split accuracy was marginally higher without it – reinforcing that hyperparameter and preprocessing decisions should be validated

by cross-validation, not a single train-test split. The practical recommendation from this experiment is to apply PCA selectively based on the downstream model family, rather than as a blanket preprocessing step.

13 References

[1] Scikit-learn Documentation, "PCA," https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html

[2] Scikit-learn Documentation, "Ensemble Methods," https://scikit-learn.org/stable/modules/ensemble.html

[3] Scikit-learn Documentation, "HistGradientBoostingClassifier," https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html

[4] UCI Machine Learning Repository, "Breast Cancer Wisconsin (Diagnostic) Data Set," https://archive.ics.uci.edu